# Dynamic Layer Routing Defense for Real-Time Embedded Vision

ANONYMOUS AUTHOR(S)

Deep neural networks have significantly advanced the perception and decision-making functions of smart embedded systems, such as car-borne driver assistance. Deploying these embedded neural networks often faces two challenges: (1) *Security vulnerabilities* to adversarial examples that can be deployed in the physical environment being perceived; (2) *limited computational resources* coupled with *dynamic conditions* that necessitate real-time adaptation of model execution. However, these two challenges are often addressed separately in existing research. This paper presents LeapNet, which aims to address both challenges simultaneously. It comprises two versions: LeapNet-1 and LeapNet-2. LeapNet-1 employs dynamic layer routing to counteract adaptive adversarial-example attacks while reducing computational redundancy. Building upon LeapNet-1, LeapNet-2 further adapts its layer routing configurations in real time to meet the requirement on frame processing rate under dynamic conditions while maintaining defense performance. Extensive experiments on various representative datasets, neural network models, and adaptive attacks demonstrate the superiority of LeapNet over existing defense methods. On-road tests with a real-time car-borne traffic sign recognition system validate its effectiveness in maintaining frame processing rate under dynamic conditions.

## 1 Introduction

Deep neural networks (DNNs) have shown good performance in various computer vision tasks, such as image classification, object detection, and tracking [1–3]. Their use in embedded and mobile applications, such as unmanned terrestrial and aerial vehicles, has attracted increasing attention [4–6]. However, despite these interests, two challenges remain in deploying them on embedded systems. First, the adoption of DNNs raises security and reliability concerns due to their known vulnerability to adversarial-example attacks [7–9]. Second, the deployment of the DNNs on embedded systems is hindered by constrained computational resources and, even more challengingly, by the complex dynamic conditions requiring real-time adaptation of model execution. These two challenges are explained in detail as follows.

*Security challenge: DNN-based embedded perception systems are vulnerable to adaptive adversarial attacks.* Embedded perception is a key component of various systems such as driving assistance agents [10, 11]. However, recent accidents related to these agents point to significant safety concerns stemming from perception errors, including failures to detect articulated buses or pedestrians [12]. According to the 2023 annual disengagement reports from the California Department of Motor Vehicles, 40% of disengagements were attributed to failures in the perception module, the highest proportion among all contributing factors [13]. Such potential unsafety undermines confidence in the core machine learning technologies underpinning modern embedded perception. Furthermore, recent studies demonstrate the fragility of DNN-based computer vision systems under the hostile setting. For instance, with adversarial perturbations resembling natural noises, computer vision models misclassify stop signs as 45-mph speed limit signs [14]. The robustness of the safety-critical embedded perception against the crafted adversarial perturbations needs to be enhanced.

A plethora of defense mechanisms have been developed to harden DNNs against the threats, such as adversarial training, input transformations, and gradient obfuscation [15–17]. However, these defenses are *deterministic*, in that the same input leads to the same processing path with no variability or stochasticity. As a result, they remain vulnerable to the more advanced *adaptive attacks*, where attackers iteratively probe the system, analyze its responses, and refine attack strategies to bypass defenses and maximize attack impacts [2]. To address adaptive attacks, *dynamic defense* approaches have recently gained research interest [18, 19]. Dynamic defenses adapt their mechanisms during inference, thereby complicating the adversary's task of crafting effective attacks.

For example, the study [18] applies randomized transformations to the inputs, while dynamic ensemble methods [2] use time-varying model ensembles to enhance robustness. However, these defenses do not consider embedded computing's main constraint from computational resources and dynamic environments. For instance, the dynamic ensemble approach [2] leads to multiplied computational overhead due to the simultaneous inferences with multiple models, making it poorly applicable to resource-constrained embedded systems.

*Efficiency and adaptability challenge: Resource constraints and dynamic conditions hinder the deployment of DNN-based functions in embedded systems.* DNNs' complex architectures and massive parameters pose challenges to deployment on embedded systems with limited resources. As DNNs often have inherent redundancy, prior studies have proposed various model tailoring techniques to facilitate DNN deployments. Pruning [20] directly reduces parameters by sparsifying the network, while quantization [21] decreases the model size and computational cost by casting floating-point values into lower-precision representations. Other approaches, such as early exit [22], layer routing [23], and width scaling [24], focus on reducing model structures. However, these approaches fall short of addressing dynamic conditions, such as changeable frame processing rate requirements and varying environments (e.g., power modes, the numbers of objects) [25, 26]. For instance, higher vehicle speeds result in longer safe stopping distances. The typical braking distance is approximately 9 meters at 40 km/h, but increases to around 67 meters at 110 km/h [27]. Therefore, it is necessary for autonomous vehicles to adjust their detection runtime according to the speed to ensure sufficient safe stopping distances [25]. In such dynamic scenarios, one-time model tailoring approaches often sacrifice accuracy to meet the strictest constraints, such as maintaining safe braking distances at maximum speed, resulting in a fixed trade-off that limits their effectiveness across different scenarios. As a relevant solution, neural architecture search (NAS) can find the optimal architectures to meet specific requirements [26, 28]. However, recent NAS-based approaches such as AdaptiveNet [26] incur long update latencies of up to minutes, making them unsuitable for real-time adaptation.

In this work, we aim to jointly address the above two challenges. To this end, we follow the *progressive system development* methodology to design two versions of a system called *LeapNet*, for resource-constrained embedded vision. The first version, LeapNet-1, focuses on the security challenge while reducing computational redundancy. The second version, LeapNet-2, further incorporates real-time adaptation to dynamic conditions.

**LeapNet-1** is a dynamic defense design based on *layer routing*. It first learns the optimal layer routing distribution and dynamically samples a route from the distribution to drop redundant layers for the given input. Specifically, we introduce a decision network for routing trained with reinforcement learning. The learning objective is to reduce the number of layers for less computational redundancy and keep the stochasticity of the dynamic layer routing while maintaining accuracy. The stochasticity increases the adversarial robustness of the target DNN model since the attackers cannot capture the inference routes for given inputs and thus have a reduced basis to construct tailored attacks.

**LeapNet-2** integrates a latency-aware component into LeapNet-1, which adapts layer routing to achieve the required frame processing rate under dynamic conditions while maintaining defense performance. Specifically, we build a latency predictor for the given embedded vision system, which can predict the runtime latency of different layer routing configurations. This facilitates the training of the decision network to develop latency awareness for learning the layer routing distributions. The average latency of all sample routes from these routing distributions can adhere to the required frame processing rate in the continual processing of frames while maintaining stochasticity to counteract adaptive attacks.

To summarize, this paper makes the following contributions:


(a) PGD attack. (b) Patch attack.

Fig. 1. Illustration of two representative adversarial attacks: PGD attack [15] and Patch attack [14].

- To the best of our knowledge, LeapNet is the first framework designed to simultaneously address the two challenges faced by embedded vision: ensuring security against adaptive adversarial attacks while managing the embedded deployments with limited computational resources and real-time adaptability under dynamic conditions.
- LeapNet-1 leverages a decision network to generate dynamic layer routes, defending against adaptive attacks while improving computational efficiency and maintaining an attainable accuracy. Furthermore, LeapNet-2 incorporates a latency-aware mechanism into LeapNet-1, adjusting layer routing to adapt to dynamic conditions for real-time embedded vision tasks.
- We evaluate the proposed LeapNet through both collected datasets and real-world outdoor experiments. The experimental results demonstrate its superior defense performance against various attacks compared with recent defense methods. Additionally, LeapNet-2 achieves real-time adaptation to varying conditions across different devices and target DNN models.

The remainder of the paper is organized as follows. Section 2 introduces the preliminaries and discusses the key observations and motivations. Section 3 and Section 4 describe LeapNet-1 and LeapNet-2, respectively. Section 5 introduces the experimental settings and discusses the experimental results. Section 6 introduces our system implementation. Section 7 discusses related works. Finally, Section 8 concludes the paper.

## 2 Background and Motivation

This section introduces the preliminaries on attacks and defenses, followed by our key observations and motivations behind the proposed LeapNet.

### 2.1 Preliminaries

■ **Adversarial Examples.** Adversarial examples refer to crafted inputs modified with small perturbations to mislead DNN with incorrect predictions [15]. Let $f(x;\theta)$ denote the classifier with weights $\theta$ and input $x$ associated with label $y$. An adversarial example $x' = x + \delta$ is tailored to mislead the classifier, such that $f(x';\theta) \neq y$, where $\delta$ is a subtle perturbation. Fig. 1 illustrates the impact of adversarial examples on pre-trained DNN models. In Fig. 1a, a digital perturbation generated using PGD algorithm [15] causes a "tiger cat" picture to be misclassified as a "lens cap". In the physical world, where pixel-level adjustments are impractical, adversaries can use adversarial patch attacks [14], as shown in Fig. 1b, causing the "stop" sign to be misclassified as the "yield" sign.

■ **Static Attacks vs. Adaptive Attacks.** A static attack refers to an attack that is designed and executed once, remaining unchanged throughout its deployment. Specifically, once launched, a static attack does not leverage any further interaction or feedback from the target model or its defense mechanisms. In contrast, an adaptive attack represents a more advanced and iterative threat model. Adaptive attackers can repeatedly interact with the target model, observe the model's

Table 1. Adversarial accuracy comparisons under diverse attacks and defense settings, in which the deterministic defense corresponds to Blockdrop [23] and the dynamic defense corresponds to the proposed LeapNet-1.

| Dataset | Defense Method | Static Attack | Adaptive Attack |
|---|---|---|---|
| CIFAR-10 | w/o Defense (Baseline) | 29.1% | 29.1% |
| | w/ Deterministic Defense | 61.7% | 35.8% |
| | w/ Dynamic Defense | 61.0% | **57.7%** |
| GTSRB | w/o Defense (Baseline) | 37.2% | 37.2% |
| | w/ Deterministic Defense | 85.6% | 22.3% |
| | w/ Dynamic Defense | 79.2% | **71.6%** |

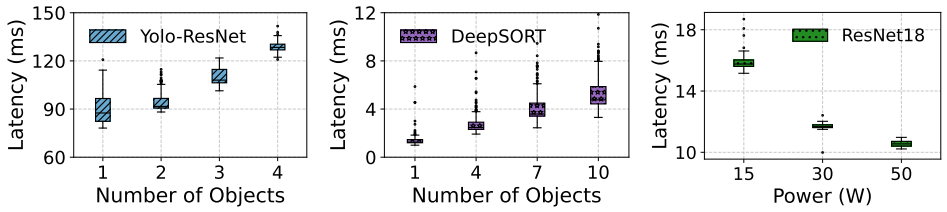
defense mechanism, and iteratively adjust its attack strategy. Such adaptability enables attackers to continuously refine their strategies, circumventing defenses by dynamically evolving their attacks.

■ **Deterministic Defense vs. Dynamic Defense.** A deterministic defense against adversarial examples refers to employing a fixed strategy to mitigate adversarial attacks. Deterministic defense always responds in the same way given a particular input, without introducing any randomness or variability. While deterministic defenses are straightforward to analyze and evaluate, they can be vulnerable to adaptive attacks due to their predictable behavior. In contrast, a dynamic defense introduces elements of randomness or variability in its response to adversarial inputs. Dynamic defense adapts or changes its behavior during inference, making it difficult for attackers to anticipate or exploit the defense mechanism during inference. Thus, dynamic defenses are more resilient to adaptive attacks, especially those relying on detailed knowledge of the defense strategy.

### 2.2 Observations and Performance Preview of LeapNet

■ **Observation 1.** *Existing deterministic defense mechanisms exhibit inferior defense performance compared with their dynamic counterparts, especially when confronting advanced adaptive attacks.* Beyond static attacks, adaptive attacks are increasingly adopted for evaluating defense performance [2]. Adaptive attacks can circumvent deterministic defenses, leading to a significant reduction in accuracy. To demonstrate this, we conduct an experiment using ResNet-18 as the target model and employing FGSM attack algorithm [7] to generate static attack and adaptive attack, on the CIFAR-10 [29] and GTSRB [30] datasets. The two datasets cover 10-class general image classification and 43-class traffic sign classification, respectively. The difference in attack effectiveness arises from the attacker's knowledge of the target model, despite using the same attack algorithm. Table 1 presents the adversarial accuracy (i.e., the accuracy under attack), before security enhancement and with enhancement by deterministic or dynamic defense. Deterministic defense refers to Blockdrop [23], a representative layer routing network that can generate a fixed inference route per input, varying across different inputs. Although it performs reasonably under static attacks, its effectiveness drops remarkably under adaptive attacks, even worse than no defense on GTSRB. The dynamic defense in Table 1 refers to LeapNet-1 proposed in this paper, which introduces stochasticity to the layer routing network. It achieves better defense performance against both static and adaptive attacks, compared with the deterministic defense.

■ **Observation 2.** *Embedded devices often operate under various dynamic conditions, where the available resources and requirements may change over time.* There have been extensive efforts toward efficient DNN deployment on embedded systems. However, these studies may not be sufficient to address the dynamics from the embedded environments [26]. We identify two types of dynamics:




(a) Object detection. (b) Object tracking. (c) Classification on Jetson Orin.

Fig. 2. Impact of dynamic conditions for embedded vision scenarios.

(1) **Varying embedded environments.** Embedded systems may operate under changing environments, such as fluctuating scene complexity and computational resources. These factors challenge the performance of DNN models, especially for time-series tasks.

*Scene complexity* refers to variations in the number and complexity of objects in input images or videos, which directly influence the inference latency of DNN models. Fig. 2a and Fig. 2b illustrate how an increase in the number of objects changes the runtimes of the object detector and tracker on a server with RTX 2080Ti GPU. DeepSORT [31] is used as the tracker, while Yolo-ResNet [2] represents a system combining YOLO for detection with ResNet18 for further classification. These results demonstrate that runtime latency grows with an increased number of detected objects.

*Fluctuations* in available computing resources, such as power supply mode, also impact the operation of embedded tasks. For example, an insufficient power supply may compel an embedded device to switch to a low-power mode, such as the automatic low-power battery switch of a drone for returning home safely [32]. When the power supply drops and switches to a low-power mode, inference latency can increase markedly, potentially violating the requirements of embedded vision tasks. Fig. 2c shows latency variants of ResNet18 [33] under different power modes, showing an approximately 50% latency increase when the power mode is switched from 50W to 15W.

(2) **Varying performance requirements.** Embedded systems often have diverse and evolving performance requirements, such as varying frame processing rates [26]. These systems need to dynamically adjust computational resources or models to maintain real-time responsiveness under different requirements. For instance, in autonomous vehicles (AVs), latency directly impacts safety-critical decisions, such as braking. Considering an AV traveling at 7 m/s, if the object detector EDet2 with a relatively shorter frame processing latency is used, a safe stopping distance of 7.66 meters is required [25]. Differently, if EDet6 with a relatively longer frame processing latency is used, a safe stopping distance of 11.14 meters is required [25]. This variation underscores the necessity for AVs to scale the complexity of vision models in response to real-time speed and braking constraints. Achieving such adaptive control is essential for balancing accuracy and latency requirements in embedded applications.

■ **Observation 3.** *Existing deterministic and dynamic defense mechanisms cannot adapt themselves toward dynamic conditions.* Effective deployment on embedded systems needs to consider limited computational resources and complex dynamic conditions. However, existing defense research, including both deterministic and dynamic defense, often overlooks these specific constraints of the practical deployment on embedded systems. Many defense methods, such as ensemble learning, enhance the security of DNNs at the expense of increased computational load. While other strategies, such as adversarial training, avoid additional inference-time costs, they often

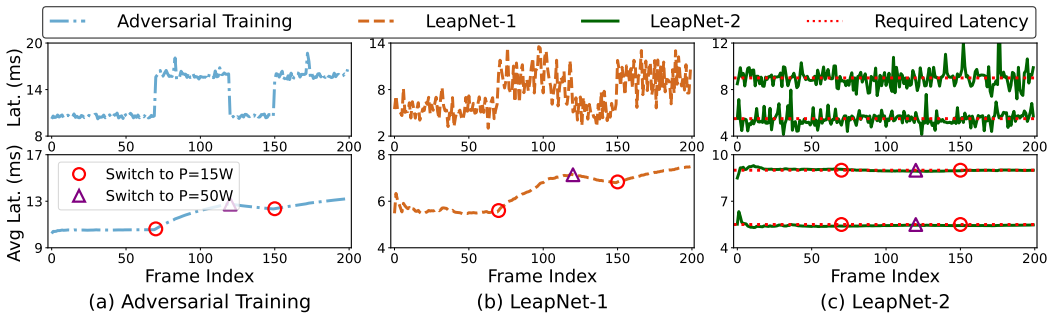



Fig. 3. Visualization of the runtime latency measured on NVIDIA Jetson Orin under different power modes.

fall short of accommodating the dynamic operational conditions encountered during embedded deployment. One direct strategy for adapting to dynamic conditions is deploying multiple security-enhanced model variants for coarse adjustment. However, this results in frequent model switching, causing overhead due to the paging-in and paging-out of entire models [26]. Thus, there is a need for a unified DNN-based framework capable of simultaneously addressing security concerns and efficiently managing computational limitations and dynamic conditions in embedded environments.

To demonstrate the above, we conduct a preliminary latency experiment evaluating the performance of various defense methods on an embedded device, Jetson Orin. Fig. 3 shows the runtime results of these defense methods applied to ResNet18 under dynamic power conditions. Compared with adversarial training, our proposed dynamic layer routing approach LeapNet-1 shows reduced inference latency. However, both adversarial training and LeapNet-1 exhibit clear latency increases when the system switches to a lower power mode, as shown by the blue and orange lines. In contrast, LeapNet-2 (green line) maintains latency closely aligned with the required value (red line) under different settings, showing LeapNet-2's capability to stabilize latency performance.

### 2.3 Motivations

On the one hand, as discussed in Observation 1, deterministic defense methods suffer from inferior defense performance against advanced adaptive attacks, which motivates us to design effective dynamic defense methods. To this end, we propose LeapNet-1 featuring dynamic layer routing mechanisms, which leverages a decision network to first explore the optimal layer routing distribution for the given input (see Fig. 4(*top*)). Furthermore, we can dynamically sample layer routes from the resulting optimal layer routing distribution, which maintains superior accuracy while also exhibiting stochasticity for adversarial robustness. This stochasticity enables LeapNet-1 to counteract adaptive attacks. To the best of our knowledge, LeapNet-1 is the first defense approach to enable dynamic layer routes.

On the other hand, as discussed in Observation 2, beyond resource constraints, dynamic conditions such as varying runtime environments and changing performance requirements challenge the embedded deployment of security-critical scenarios. Existing security-enhanced DNNs cannot adapt to real-world dynamic embedded conditions, especially those time-series embedded vision tasks (e.g., video processing). To this end, we draw insights from LeapNet-1 and further introduce LeapNet-2 (see Fig. 4(*bottom*)). Specifically, LeapNet-2 features a latency predictor that enables the decision network to learn the latency of different layer routes. Then, this latency-aware decision network can explore the optimal layer routing distribution that satisfies the specified,

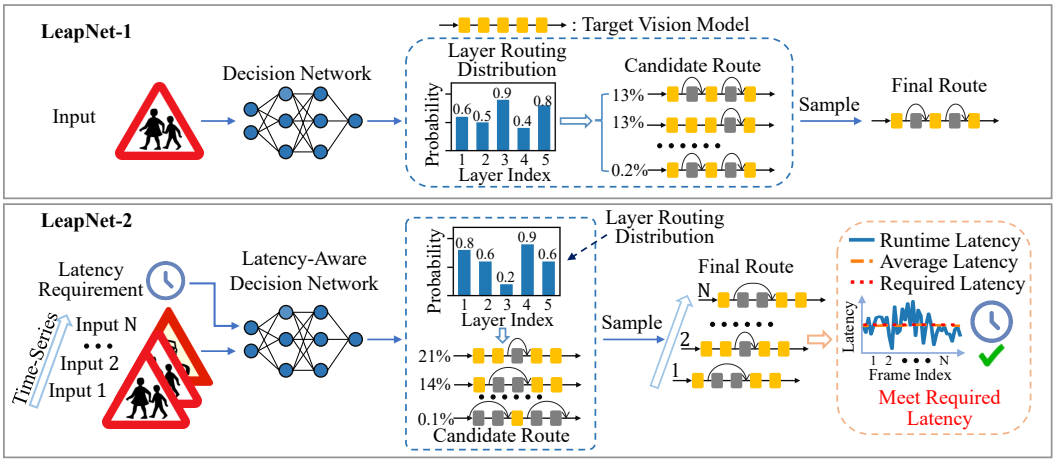



Fig. 4. Overview of LeapNet, where (1) LeapNet-1 focuses on exploring the optimal layer routing distribution towards robust defense performance against adaptive attacks while reducing computational redundancy and (2) LeapNet-2 focuses on exploring the optimal layer routing distribution that satisfies the specified latency constraint under dynamic conditions while maintaining defense performance.

adjustable latency constraint. Furthermore, dynamically sampled layer routes operate near the latency constraint, allowing the average latency to adapt to the latency constraint in real time.

Taken the above together, the proposed LeapNet can effectively defend against adaptive attacks while reducing computation redundancy. Furthermore, LeapNet can achieve real-time adaptation to dynamic conditions in embedded vision systems while maintaining defense capability.

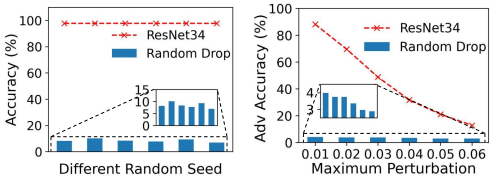
## 3 LeapNet-1

In this section, we introduce LeapNet-1, which employs a decision network to explore the optimal layer routing distribution. Furthermore, the dynamic layer routes sampled from the resulting distribution can effectively defend against adaptive attacks while reducing computational redundancy.

### 3.1 Preliminaries on Target Model

As shown in Fig. 4, LeapNet leverages a decision network to generate dynamic layer routes for the target model. Following recent conventions [23, 26], we take ResNet [33] as the default target model, which is a widely adopted basis for vision tasks due to its effectiveness and simplicity. ResNet utilizes shortcut connections to add the input to the output of the residual block. Although each residual block is composed of multiple layers, for simplicity, we treat each residual block as a single layer and thus use the term *layer* instead of *block* when describing the routing process. Similar layer routing mechanisms can be extended to other DNN architectures through gating mechanisms with minor revisions [34].

We use $x$ and $y$ to denote the input image and its ground-truth label and consider a target DNN model $f_C(\cdot)$ with $N$ layers. Finally, we can formulate the forward propagation of $f_C(\cdot)$ as $y' = f_C(x; \phi_C)$, where $\phi_C$ denotes the weights of $f_C(\cdot)$ and $y'$ denotes the predicted result. Furthermore, the training process optimizes $\phi_C$ to minimize the cross-entropy loss on the training dataset: $\text{minimize}_{\phi_C} J_C = L_C(f_C(x; \phi_C), y)$, where $L(\cdot, \cdot)$ denotes the cross-entropy loss.




(a) Clean accuracy. (b) Adversarial accuracy.

Fig. 5. Accuracy of ResNet34 with and without random layer dropping on KUL dataset. Maximum Perturbation indicates the perturbation upper bound.

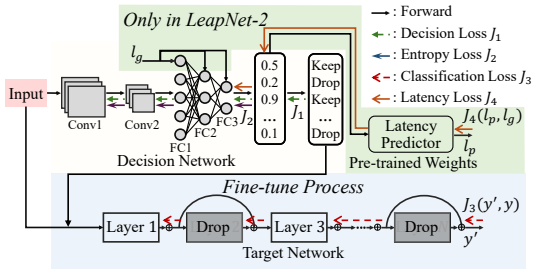



Fig. 6. The proposed training algorithms of LeapNet.

### 3.2 Decision Network

As discussed in [23], DNNs are redundant, allowing some layers to be dropped with minimal accuracy loss. However, randomly dropping layers may degrade accuracy. To demonstrate this, we conduct a preliminary experiment using ResNet34 on the KUL dataset [35]. As shown in Fig. 5, applying a random layer dropping strategy yields only 7.0%~10.1% accuracy on clean inputs and 3.0%~3.9% accuracy on PGD-attacked inputs. In contrast, ResNet34 without layer dropping can have a 97.7% clean accuracy and 88.1% to 12.7% accuracy under PGD attacks as the maximum perturbation bound increases from 0.01 to 0.06 (relative to the maximum input pixel value).

Despite the acknowledged redundancy, determining which layers are redundant remains challenging. Given a target DNN with $N$ layers, the number of possible layer routes grows exponentially (i.e., $2^N$), making exhaustive exploration impractical. To address this issue, we propose a reinforcement learning-based decision network capable of simultaneously deriving layer routings across all layers. It can learn the optimal layer routing distribution without the labeled data, thereby eliminating the computational burden associated with individually evaluating each routing combination. The decision network features a lightweight DNN model comprising two convolutional and three fully-connected layers. Specifically, the decision network outputs a *probs vector*, $p = [p_1, ..., p_n, .., p_N]$, where $p_n \in [0, 1]$ is the probability of executing the $n$-th layer. Then, the inference routes are sampled based on this vector. In this case, the decision for the target network with $N$ layers that is dropped or executed would follow the Bernoulli distribution:

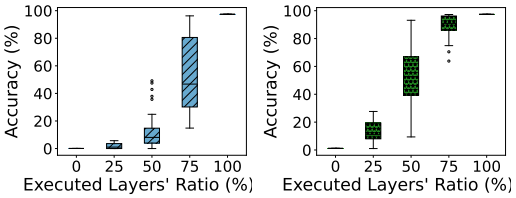
$$\pi_{\phi_D}(a|x) = \prod_{n=1}^{N} p_n^{a_n}(1-p_n)^{1-a_n}, \text{ s.t., } p = f_D(x;\phi_D), \tag{1}$$

where $f_D(\cdot)$ denotes the decision network and $\phi_D$ denotes its weights. Correspondingly, $a = [a_1, ..., a_n, ..., a_N]$ represents the action obtained through Bernoulli sampling according to probability $p$. The $a_n \in \{0, 1\}$ represents the action of either dropping ($a_n = 0$) or executing ($a_n = 1$) this layer.

defense details...

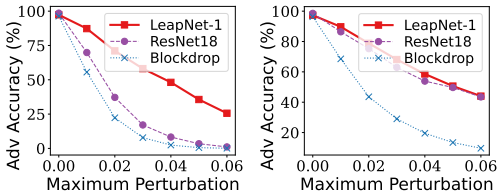
The optimization objectives of LeapNet-1 are three-fold, including (1) maximizing the accuracy on clean inputs, (2) maximizing the number of dropped layers for less computation, and (3) maximizing the stochasticity against adaptive attacks. To this end, we design the reward function $R(\cdot)$ and introduce a decision loss to optimize the objectives (1) and (2). Note that an entropy loss is further added to optimize the objective (3) as shown in Equation (4). Specifically, the reward function $R(\cdot)$ is formulated as follows:

$$R(a, \phi_C, x, y) = \begin{cases} 1 - \left(\frac{\|a\|_1}{N}\right)^2 = 1 - \left(\frac{\sum_{n=1}^{N}|a_n|}{N}\right)^2, & \text{if } f_C(x;\phi_C(a)) = y, \\ -\gamma, & \text{otherwise,} \end{cases} \tag{2}$$




(a) Before fine-tuning. (b) After fine-tuning.

Fig. 7. Accuracy vs. executed layers' ratio of ResNet18 on the KUL dataset before and after fine-tuning.




(a) PGD attack. (b) FGSM attack.

Fig. 8. Adversarial accuracy comparisons under different attacks on GTSRB dataset.

where $\frac{\|a\|_1}{N}$ quantifies the number of activated layers. $\|\cdot\|_1$ is the Manhattan $l_1$ norm. $f_C(x;\phi_C(a))$ is the classification model with layer route action $a$. The $\gamma$ is used to penalize the decision that produces incorrect predictions, which can trade-off between accuracy and the number of activated layers. Finally, we formulate the decision loss as the negative objective of the reward function $R(\cdot)$ and minimize the decision loss to maximize the reward as follows:

$$\begin{aligned} J_1 &= -R(a,\phi_C,x,y)\log\pi_{\phi_D}(a|x) \\ &= -R(a,\phi_C,x,y)\sum_{n=1}^{N}\left(a_n\log p_n + (1-a_n)\log(1-p_n)\right), \end{aligned} \tag{3}$$

Moreover, the entropy loss to maximize the decision stochasticity to counteract adaptive attacks is:

$$J_2 = -\sum_{n=1}^{N}(p_n\log p_n + (1-p_n)\log(1-p_n)), \tag{4}$$

To summarize, the decision and entropy loss objectives in Equation (3) and (4) can optimize the decision network to explore dynamic layer routing solutions to counteract adaptive attacks while maintaining good accuracy and computational efficiency.

### 3.3 Training & Fine-tuning

LeapNet-1 involves two separate stages, i.e., training and fine-tuning. Specifically, the training stage only optimizes the decision network, while the fine-tuning stage optimizes both the decision network and the target model.

**Stage 1: Training.** During training, we aim to optimize the decision stochasticity while maximizing the number of dropped layers and the classification accuracy of the target model. Specifically, the decision network is trained with the following loss function:

$$J_{D_1} = \mathbb{E}_{\phi_D,x,y}[J_1(\phi_C,\phi_D,x,y)] + \alpha\cdot\mathbb{E}_{\phi_D,x}[J_2(\phi_D,x)], \tag{5}$$

where $\alpha$ is a constant to control the level of stochasticity. Note that this stage only optimizes the decision network's weights $\phi_D$, while the pre-trained target model's weights $\phi_C$ are frozen.

**Stage 2: Fine-tuning.** The above layer-routing scheme inevitably suffers from inferior accuracy compared with the default target model [23], primarily due to the unavoidable omission of certain useful layers. To mitigate this performance degradation, we further adapt the target model to the layer routing behaviors learned from the decision network. Specifically, as shown in Fig. 6, we fine-tune both the target model and the decision network to optimize the following loss function:

$$J_{D_2} = \mathbb{E}_{\phi_D,x,y}[J_1(\phi_C,\phi_D,x,y)] + \alpha\cdot\mathbb{E}_{\phi_D,x}[J_2(\phi_D,x)] + \beta\cdot\mathbb{E}_{\phi_C,\phi_D,x,y}[J_3(\phi_C,\phi_D,x,y)], \tag{6}$$

Table 2. Inference Cost Comparisons.

| Model | Runtime (s) | Static Memory (MB) | Runtime Max Memory (MB) |
|---|---|---|---|
| ResNet18 | 10.6 | 42.7 | 186.4 |
| Decision model | 1.3 | 0.7 | 47.6 |
| LeapNet-1 | 6.8 | 43.4 | 185.6 |

Table 3. Training Cost Comparisons.

| Method | Phase | Runtime Max Memory(MB) | Runtime(s)/Epoch | Epoch Number | Total Runtime (s) |
|---|---|---|---|---|---|
| Adversarial Training | | 951 | 57.7 | 20 | 1154 |
| LeapNet | ResNet18 Training | 677 | 22.9 | 16 | 817.9 |
| | Decision Training | 492 | 12.6 | 10 | |
| | Fine-tune | 732 | 21.7 | 15 | |

where $\alpha$ and $\beta$ are the coefficients to control the trade-off magnitude between $J_1$, $J_2$, and $J_3$. Here, $J_3$ corresponds to the classification loss for the target model based on the routes generated from the decision network, which can be described as follows:

$$J_3 = L_C(f_C(x; \phi_C(a)), y), \tag{7}$$

**Cost Analysis:** The cost of LeapNet comprises two components: inference cost and training cost. *(1) Inference Cost:* The proposed decision network contains 0.17 million parameters, in contrast to the smallest target model, ResNet18, which has 11.2 million parameters. This corresponds to a static memory footprint of merely 0.67 MB for the decision network, compared to 42.7 MB for the target model ResNet18. We benchmark the inference latency and memory usage of ResNet-18 and LeapNet on a Jetson Orin device, processing 100 continuous image frames, as shown in Table 2. The decision model incurs an additional latency of approximately 12.3% relative to ResNet-18. This is attributed to the kernel launch overhead on Jetson Orin, which becomes more significant for small models with low GPU utilization. Despite this, LeapNet-1 achieves a 35.8% reduction in average inference latency, demonstrating its effectiveness in mitigating computational redundancy in the target model. Although the decision model consumes additional memory during inference, the overall runtime memory of LeapNet-1 remains comparable to that of ResNet-18. This is because the dominant contributor to inference-time memory usage in CNNs is feature map storage. LeapNet-1 selectively skips layers, thereby avoiding the computation and storage of many intermediate feature maps. As a reference, the runtime peak memory usage of ResNet-18 with all selectable layers (intermediate feature maps) dropped is as low as 47.1 MB. *(2) Training Cost:* We also compare the training time and peak memory usage of LeapNet and adversarial training (with FGSM attack) on a server equipped with an RTX 2080 Ti GPU, as shown in Table 3. The training process of LeapNet consists of three phases: (i) training the target model (ResNet-18), (ii) training the decision model, and (iii) joint fine-tuning. Despite the three-stage training process, LeapNet demonstrates lower training time and memory usage compared to the adversarial training baseline. This is primarily because adversarial training requires the generation of perturbed inputs based on model parameters, which introduces additional computational overhead. These results highlight the training efficiency of LeapNet.

**Results.** *(1) Accuracy:* We conduct an experiment on the KUL dataset [35] to demonstrate the effectiveness of fine-tuning. The results in Fig. 7 depict the relationship between accuracy and the executed layers' ratio of ResNet18 before and after fine-tuning. The results show that the joint fine-tuning stage can improve the accuracy across all executed routes of the target model

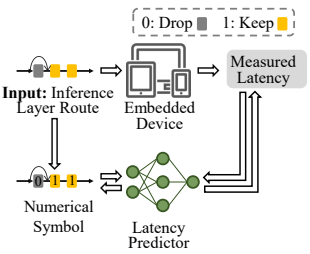



Fig. 9. The training process of latency predictor with layer routes as input.

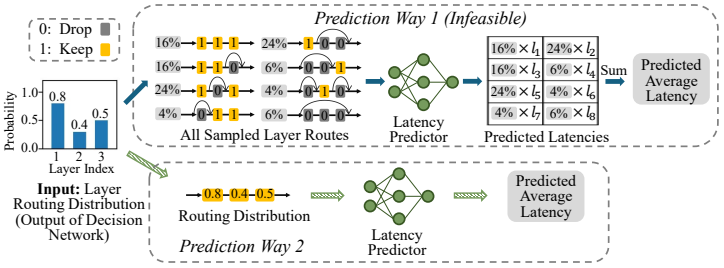



Fig. 10. Two different prediction ways during LeapNet-2's training: (1) Prediction by inputting all sampled routes from layer routing distributions. (2) Prediction by inputting layer routing distributions.

ResNet18. Besides, during fine-tuning, the decision network is also optimized to learn the layer routes with better accuracy. This joint optimization of both the target model and decision model enables LeapNet-1 to achieve good accuracy. *(2) Adversarial Performance:* We compare LeapNet-1, Blockdrop [23], and the original ResNet18 in terms of adversarial robustness against adaptive attacks. Fig. 8 shows the results on the GTSRB dataset. LeapNet-1 outperforms ResNet18 and Blockdrop against adaptive attacks based on the representative attack algorithms of PGD [15] and FGSM [7] with a range of adversarial perturbation levels.

## 4 LeapNet-2

In this section, we introduce LeapNet-2, an enhanced version that incorporates latency-awareness into the decision-making process. Specifically, LeapNet-2 first builds a latency predictor to estimate the inference latency of the target model across different layer routes. With this latency predictor, LeapNet-2 trains a latency-aware decision network to produce the optimal layer routing distribution that satisfies the specified latency constraint. Consequently, the layer routes sampled from the resulting distribution consistently operate around the given latency setpoint, enabling the average latency of sampled routes to rapidly converge toward the setpoint. Note that the latency setpoint can be adjusted at run time.

### 4.1 Latency Predictor

First, we construct a latency predictor tailored for inputting layer routes of the target model. We train the predictor with collected data by measuring the inference latency of these different layer routes on the target hardware, as explained by Fig. 9. For a small target model, we measure the latency for all possible layer routes multiple times for constructing the dataset. However, for a large target model (such as ResNet101 with 34 blocks), the routing space is large (e.g., $2^{34}$ decisions). To deal with this, we use stratified sampling and random sampling techniques to select representative routes from this large routing space. Then, we build a latency predictor to model the relationship between inference latency and layer routes, described as $l_p = f_P(a; \phi_L)$, where $l_p$ denotes the predicted latency, $f_P(\cdot)$ denotes the latency predictor, and $\phi_L$ denotes its weights.

We consider five representative prediction methods from the *scikit-learn* library [36], including MLP regression, linear regression, SVM regression, SGD regression, and kernel regression. Taking RTX2080 Ti as an example target device, we visualize the latency prediction results of ResNet18 in Fig. 11. Among the above five prediction methods, we observe that the MLP regression algorithm can achieve the lowest root mean squared error (RMSE), demonstrating superior latency prediction performance across various layer routes of ResNet18.

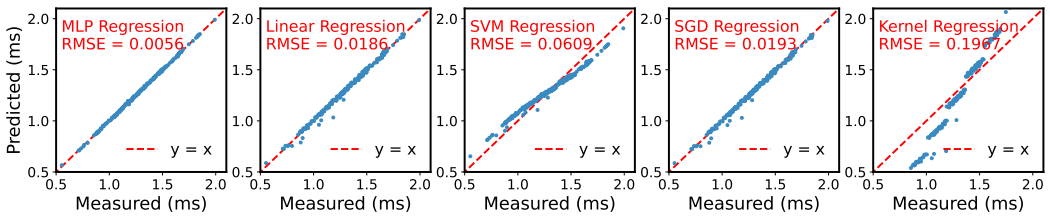



Fig. 11. Latency prediction performance of different regression algorithms under different layer routes.

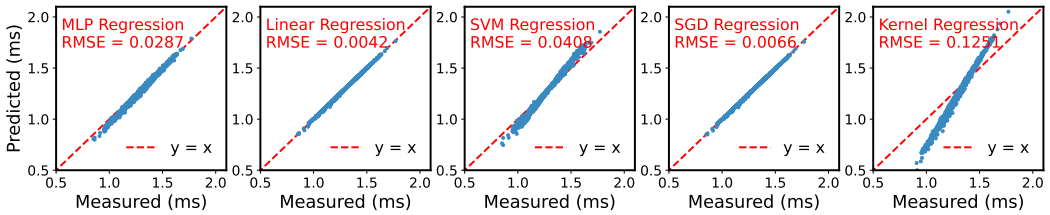



Fig. 12. Latency prediction performance of different regression algorithms (trained with inputting layer routes) under layer routing distributions. "Predicted" and "Measured" refer to the average latency of all sampled routes from the distribution.

To enable LeapNet-2's decision network with latency-awareness, it needs to learn the latency information of the target model across different layer routes from the latency predictor. Since the decision network learns the layer routing distribution, a straightforward approach is to input the sampled routes from the learned layer routing distribution to the pre-trained predictor to estimate the latencies of these sampled routes and then calculate the average latency during LeapNet-2's training stage, as shown in *prediction way 1* in Fig. 10(top). However, the sampling process is non-differentiable, which disrupts gradient flow and prevents the decision network from converging, making *prediction way 1* infeasible.

Therefore, the only feasible approach for LeapNet-2's training is to use a latency predictor that directly estimates the average inference latency of all sampled routes by taking the layer routing distribution as input, as shown in *prediction way 2* in Fig. 10(bottom). However, directly training such latency predictor is infeasible due to the need to consider all possible sampled routes when calculating the latency of this distribution during the training phase. Performing this computation for each layer routing distribution would incur substantial computational overhead, making the process impractical.

We anticipate that the pre-trained latency predictor designed for inputting layer routes can be extended to handle layer routing distributions (*prediction way 2*), thereby avoiding the computationally intensive process of directly learning from layer routing distributions. This extension is feasible since we observe that linear regression performs well in latency prediction, indicating an inherent nearly linear relationship between routing decisions and inference latencies, from the results in Fig. 11. According to Jensen's inequality, in a linear relationship, the predicted average inference latency $f_P(p;\phi_L)$ under a layer routing distribution $p$ is equal to the weighted sum of the predicted latencies of all sampled routes $f_P(a;\phi_L)$ from this routing distribution.

We conducted an experiment to evaluate whether pre-trained latency predictors designed for inputting layer routes remain effective when applied to layer routing distributions. Specifically, we input the routing distributions $p$ into the predictor to derive the average latency in prediction $f_P(p;\phi_L)$. Next, we input all possible sampled routes $a$ from the distribution into the predictor to

derive the latency for each route $f_P(a; \phi_L)$. Then, we compare the average latency in prediction to the weighted average of all possible routes. The detailed results are shown in Fig. 12, where the linear regression-based latency predictor achieves the best performance. When inputting layer routes into the pre-trained predictors, the MLP regression method achieves less prediction error (Error 1), measured by RMSE. However, when inputting layer routing distributions, the MLP regression introduces an additional error (Error 2) due to its non-linear characteristics, as suggested by Jensen's inequality. In contrast, the linear regression method, being free from non-linear characteristics, avoids this additional error (Error 2), making it the best choice for predicting the average latency of layer routing distributions. Thus, we choose the linear regression algorithm as the latency predictor under stochastic routing decisions.

### 4.2 Latency-Aware Decision Network

To enable latency-awareness in the decision network of LeapNet-2, latency information must be explicitly incorporated into the decision network. To this end, we first integrate a latency branch within the decision network to receive the latency requirements as input. Directly incorporating latency into the input challenges the decision network to learn latency requirements, as the input image is far more complex than the single latency value. To address this, we integrate latency information before multiple fully connected layers, enhancing its significance. Furthermore, we leverage a pre-trained latency predictor to facilitate the training of the decision network: the decision network learns to estimate the expected latency based on its outputted layer-routing distribution, allowing it to adjust its parameters accordingly through backpropagation, shown in Fig. 6. Note that the latency predictor is only required in the training process. With the above design, LeapNet-2 is capable of generating layer-routing distributions conditioned on the given latency requirements. Specifically, the weighted average of the predicted latencies across all sampled layer routes, derived from routing distribution, aligns closely with this specified latency requirement. Consequently, during continuous time-series image processing, LeapNet-2 ensures that the average inference latency consistently meets the provided latency constraints.

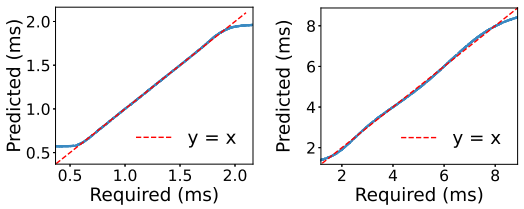
Compared with LeapNet-1, the additional objective for LeapNet-2 is to aim for the predicted average latency of all routes sampled based on the outputted layer routing distribution of the decision network to meet the latency requirements. To achieve this objective, during the training stage, the layer routing distributions are sent to the latency predictor in the latency branch to estimate the predicted average latency. Moreover, a latency loss function is designed to minimize the difference between the predicted average latency and the given latency requirement, defined by the Mean Squared Error (MSE) loss:

$$J_4 = L_{MSE}(f_P(p; \phi_L); l_g), \tag{8}$$

where $l_g$ denotes the given latency requirements. Based on this loss function, the predicted average inference latency for the layer routes derived by this latency-aware decision network closely aligns with the required latency. Given the predictor's effectiveness in matching predicted latency with measured latency, LeapNet-2 can enable real-time adjustments of layer routes to meet the dynamic conditions of embedded deployment.

### 4.3 Latency-Aware Training & Fine-tuning

As shown in Fig. 6, the training phase of LeapNet-2, similar to LeapNet-1, consists of two stages: first, training the decision network with the target model's weights fixed, and second, jointly fine-tuning both the decision network and the target model. Note that the latency predictor is pre-trained, and its weights remain fixed during both training and fine-tuning stages.




(a) ResNet18. (b) ResNet101.

Fig. 13. Illustration of the relationship between the required latency and the predicted inference latency for LeapNet-2 across different target models on RTX 2080Ti, including ResNet18 and ResNet101.

The first training stage consists of two update steps. First, similar to LeapNet-1, we train the main branch of the decision model to minimize the loss $\mathbb{E}_{\phi_D,x,y}[J_1(\phi_C,\phi_D,x,y)] + \alpha \cdot \mathbb{E}_{\phi_D,x}[J_2(\phi_D,x)]$, for defense objectives while preserving accuracy. In the second step, we train only the latency branch to minimize the latency loss $J_4$. During this process, the weights before the latency input of the decision network are frozen. This second step can be repeated multiple times to balance performance between the latency objective and other objectives.

Finally, we jointly fine-tune the target model with the decision network to improve the accuracy performance while maintaining other objectives by adapting the target model to the learned latency-aware layer routing behavior. Note that the latency requirements are randomly generated based on the measured range of the target model with different possible routes so that LeapNet-2 can adapt to variable latency requirements.

In conclusion, based on LeapNet-1, LeapNet-2 additionally considers the dynamic conditions in practical embedded scenarios. LeapNet-2 first employs a latency predictor to estimate the latencies for dynamic routes. Then, we use a latency-aware decision network to adjust layer routing distributions flexibly to adapt to dynamic embedded conditions in real time.

**Results.** Fig. 13 shows the predicted latency of the layer routes generated by the proposed latency-aware decision network, compared with given latency requirements for target models ResNet18 and ResNet101 on an edge device with RTX 2080Ti GPU. We can see that the predicted latency can match the given latency requirements closely for all target models. Furthermore, given the effectiveness of the latency predictor in aligning predicted latency with measured latency (shown in Fig. 11), the final average inference latency closely adheres to the latency requirements.

## 5 Experiment

In this section, we evaluate LeapNet to demonstrate its adversarial robustness and adaptability to dynamic conditions.

### 5.1 Experiment Setup

**Environments.** We implement our method using Python with PyTorch and conduct experiments on an edge server with Nvidia RTX2080Ti with 11GB memory, and two embedded devices, a Jetson Orin with 64 GB memory and a Jetson Xavier with 16G memory.

**Datasets and Models.** We evaluate LeapNet on two datasets: German Traffic Sign Recognition Benchmark (GTSRB) dataset [30] and KUL Belgium Traffic Signs (KUL) Dataset [35]. We experiment with three different sizes as the target model for layer routing: ResNet18, ResNet34, and ResNet101.

**Attacks.** We consider five typical attack algorithms to construct adaptive attacks in evaluating defense mechanisms, FGSM [7], PGD [15], BIM [37], Square attack [38], Autoattack[39]. We use

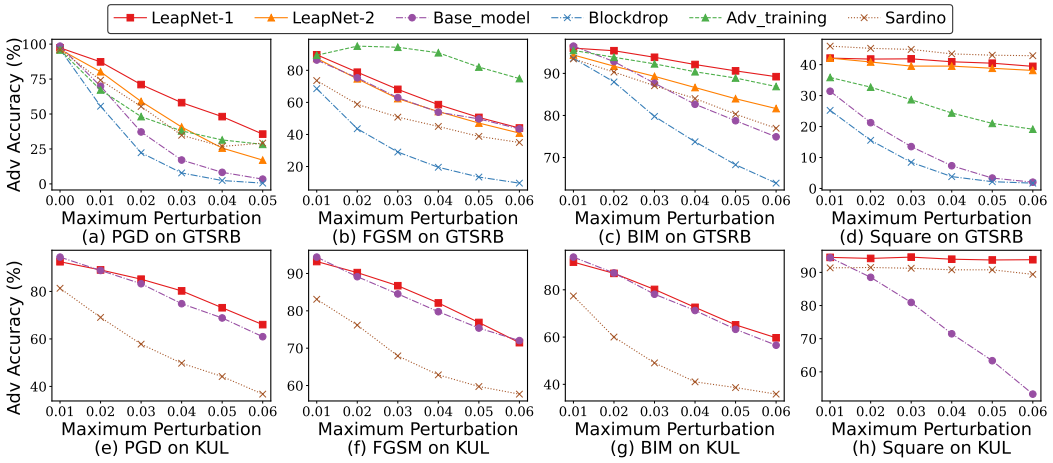


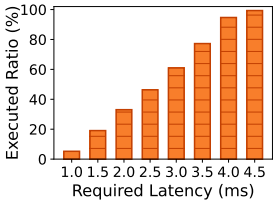

Fig. 14. Comparisons of adversarial accuracy under different adaptive attacks on GTSRB and KUL dataset.

adversarial accuracy as the defense metric, defined as the classification accuracy on adversarial examples.

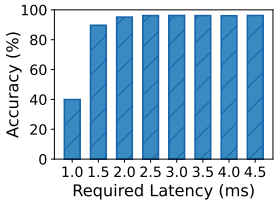
**Baselines.** We evaluate the performance of LeapNet with multiple baselines, including a no defense method, two deterministic defense methods, and two dynamic defense methods. Base_model is the target model of LeapNet. Blockdrop [23] is a representative layer routing mechanism that can switch different inference paths for different inputs, which can be seen as a deterministic defense. Adv_training [15] is a common and effective static defense method, adopting both real data and data generated by an adversary to improve its robustness. It employs the FGSM-based adversarial examples in the training stage. BaRT [18] is a dynamic defense method, randomly leveraging a combination of input transformation methods as a robust defense. Note that since many transformations are non-differentiable, we choose to directly attack the transformed images rather than performing gradient-based attacks through these transformations. This can be seen as a partially adaptive attack, since constructed attacks may be inherently weaker since it does not exploit gradient information from the entire defense mechanism. As a result, the generated adversarial examples may be less effective and overestimate BaRT's robustness. Sardino [2] is a recent dynamic defense method, adopting hypernet to generate an ensemble of classification models with high-rate ensemble renewal.

### 5.2 Performance on Adversarial Defense

We first evaluate the defense performance of LeapNet with multiple baselines under various adaptive attacks across two datasets, shown in Fig. 14. Specifically, in Fig. 14a to Fig. 14d, LeapNet-1 and LeapNet-2 achieve higher accuracy than unhardened Base_model across different perturbation settings, showing enhanced defense ability. Compared with Blockdrop, a representative layer routing method, LeapNet-1 and LeapNet-2 demonstrate superior adversarial accuracy under all attacks with a maximum 37.7% accuracy improvement. This advantage arises from the introduction of randomness in layer routing decisions, complicating attack construction. Although Adv_training surpasses LeapNet models under the FGSM attack due to using the same attack during training, it lacks generalizability to unknown attack types common in real-world applications. In contrast,




(a) Executed layers' ratio vs. Required latency.




(b) Classification accuracy vs. Required latency.

Fig. 15. Accuracy and executed layers' ratio to total layers in LeapNet-2 under different latency requirements.

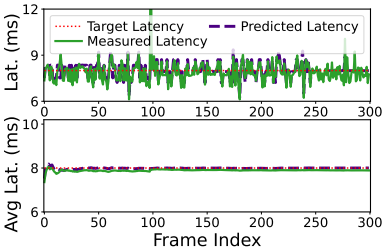



Fig. 16. Runtime and average latency of continuous frames for LeapNet-2 under different power settings on Jetson Orin.

LeapNet-1 and LeapNet-2 offer general defense performance across different attacks, outperforming Adv_training across other attacks.

Furthermore, LeapNet-1 outperforms Sardino under most attacks across both datasets. This advantage primarily stems from Sardino's design limitations, which restrict its applicability to small DNNs. Specifically, Sardino dynamically regenerates network weights to enhance robustness, but the overhead grows exponentially with network complexity, hindering its effectiveness on deeper architectures like ResNet18. For a fair evaluation, we adopt Sardino's original setup using a smaller three-layer baseline network. However, smaller networks inherently provide weaker adversarial defenses, reducing Sardino's effectiveness against strong adaptive attacks. Besides, LeapNet-2 shows similar or slightly lower accuracy compared to LeapNet-1 (0.4%∼10.9%), due to sacrificing some stochasticity in exchange for adaption to dynamic conditions.

Additionally, LeapNet is compatible with static defense methods for better robustness as shown in Fig. 14e to Fig. 14h. Specifically, these figures use ResNet34 as the target model on the KUL dataset to illustrate LeapNet's compatibility with Adv_training. In these figures, Base_model refers to ResNet34 trained with FGSM adversarial training, and LeapNet-1 is trained using the same strategy. LeapNet-1 achieves superior adversarial accuracy under various attacks, improving the overall defense capability. LeapNet-1 can provide up to a 40.6% accuracy improvement over the model trained solely with adversarial training under the Square attack.

### 5.3 Performance on Clean Accuracy

We also test the clean accuracy by our methods and the baselines, shown in Fig. 14a. When the maximum adversarial perturbation is set to 0, it indicates that no adversarial perturbation is applied, allowing us to assess the models' performance on clean data. The original ResNet18 on the GTSRB dataset achieves a test accuracy of 98.5%. In comparison, LeapNet-1 and LeapNet-2 attain accuracies of 97.3% and 96.2%, respectively, reflecting only a marginal decrease relative to the original ResNet18. This slight drop can be attributed to the model's emphasis on learning more generalized features, rather than solely optimizing for performance on clean data. A similar trend is observed in Blockdrop and Adv_training, which exhibit 96.4% and 95.8% accuracy, further supporting this notion of the minor trade-off in clean accuracy for robustness-enhancing techniques.

We also evaluate the accuracy of LeapNet-2 with ResNet34 as the target model on the KUL dataset under various latency requirements, as shown in Fig. 15. When the latency constraint is stringent, such as 1 ms, all selectable layers are dropped, leading to severe performance degradation, with accuracy dropping to just 40%. However, even a small relaxation of the latency constraint allows for significant improvements—executing only 20% of the layers boosts accuracy to 90%,

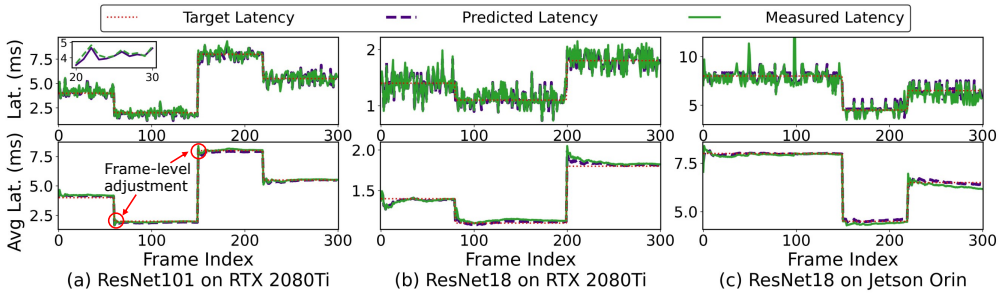



Fig. 17. Runtime and average latency of continuous frames for LeapNet-2 under different settings.

while performance stabilizes at 96% when 35% of the layers are retained for inference. These results highlight the effectiveness of dynamic layer routing in balancing efficiency and accuracy, demonstrating that selectively dropping layers can significantly reduce computation costs while preserving the model's accuracy. To maintain an acceptable level of accuracy in embedded systems, a minimum latency requirement can be enforced. This requirement can be adjusted based on the system's needs to ensure that a sufficient number of layers remain active for reliable performance.

### 5.4 Performance on Adaptability

**Varying embedded environments.** We evaluate LeapNet-2's adaptability to dynamic embedded environments with varying computing resources by altering the power mode on Jetson Orin. The power supply transitions through three stages: initially operating at 50W, then reducing to 30W after processing 100 frames, and further dropping to 15W after processing 250 frames. Fig. 16 depicts the measured, predicted, and target latency per frame (top) and their corresponding average latencies over time (bottom), computed after each latency change. The measured latency closely aligns with the predicted latency, showing the effectiveness of the latency predictor. The measured latency fluctuates around the target latency, reflecting the inherent stochasticity in our dynamic defense mechanism. Despite significant power reductions, LeapNet-2 consistently maintains average latency near the target, highlighting its stable performance across resource variations.

**Varying performance requirements.** We also test the inference latency of LeapNet-2 when processing continuous frames under varying latency requirements, as shown in Fig. 17. To test the generalizability, we considered three different scenarios: ResNet101 as the target model on RTX 2080Ti, ResNet18 on RTX 2080Ti and Jetson Orin. The measured latency closely matches the predicted latency across various model sizes and embedded devices under different latency constraints, demonstrating the accuracy and effectiveness of the latency predictor. When the given latency requirement changes, the average measured latency quickly adjusts to meet the new requirement, with only a brief period of fluctuation.

We further evaluate the adjustable latency range of LeapNet-2 across different target models and embedded edge devices, as illustrated in Fig. 18. Specifically, we deploy LeapNet-2 with ResNet18, ResNet34, and ResNet101 as target models on an edge server with an RTX 2080Ti, as well as on Jetson Orin and Jetson Xavier, which exhibit progressively lower computational capabilities. The results show that the inference latency spans from as low as 0.5 ms to nearly 80 ms, highlighting LeapNet-2's remarkable adaptability in dynamically adjusting computational complexity to accommodate diverse hardware constraints.

In conclusion, LeapNet-1 can provide robust and universal defense performance under various adaptive attacks while maintaining superior accuracy. LeapNet-2 further introduces the capability

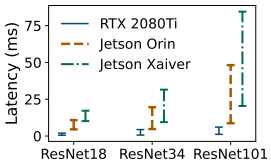



Fig. 18. Adjustable latency range for various models on different devices.

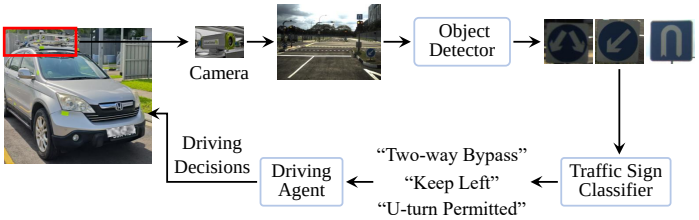



Fig. 19. Pipeline of traffic sign recognition-assisted driving systems.

to adapt to varying embedded conditions across different edge devices and target DNN models. By effectively considering computational efficiency and model robustness, LeapNet-2 proves highly suitable for security-critical embedded applications that operate under dynamic conditions.

## 6 Real-Time On-Vehicle Recognition

### 6.1 System Implementation

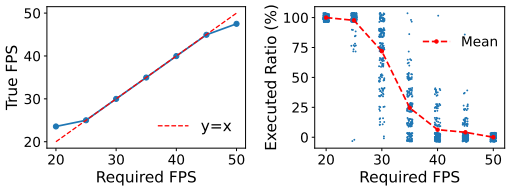
We apply LeapNet to an embedded vision system tailored for traffic sign recognition in a self-driving scenario, as illustrated in Fig. 19. A traffic sign recognition system comprises sign detection and further classification phases [2]. First, images captured by the onboard camera are buffered and passed to the detector. Then, detected traffic signs are cropped, resized, and sent to a classifier to determine their type. The results are provided to the driving agent for real-time decision-making. Given a latency constraint $t_c$, if detection takes $t_d$ seconds for a frame containing $n$ signs, the soft deadline for classifying each sign is $\frac{t_c - t_d}{n}$ seconds. We implemented and tested this system on a platform with an Alvium G1-510c for the on-broad camera and a Jetson Orin for computing.

In this system, we utilize YOLO and YOLO-tiny [40] as traffic sign detectors and LeapNet-2 as the classifier. YOLO is a representative DNN-based object detection network that achieves good accuracy and latency performance on embedded systems. YOLO-tiny is a lightweight version of YOLO with less computational requirement. The detector model is trained based on the publicly available COCO dataset [41] and KUL Belgium Traffic Signs Detection dataset [35], replacing the "stop sign" class with "traffic sign" to enhance the ability to detect traffic signs. Besides, we train LeapNet as the classifier model with the KUL Belgium Traffic Signs Classification dataset.

*Accuracy performance:* Compared to the original YOLO and YOLO-tiny with mAP@0.5 scores of 57.9 and 33.1 [40], the retrained YOLO and YOLO-tiny achieve improved mAP@0.5 scores of 58.9 and 33.7, showing the effectiveness of detecting traffic signs. mAP@0.5 denotes the mean average precision, considering predictions positive if their Intersection over Union (IoU) is at least 0.5. LeapNet, as the classifier, achieves a clean accuracy of 96.3%, comparable to the accuracy of 97.0% reported in [35]. *Latency performance:* YOLO achieves throughputs of 23 frames per second (FPS) and 15 FPS at input frame sizes of 544×608 and 704×832, the same resolutions. LeapNet, using ResNet18 as the target model, achieves inference latency ranging from 4.5 ms to 10.8 ms.

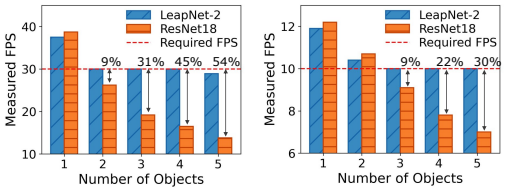
### 6.2 Performance Evaluation

In this pipeline recognition system, the dynamics arise from two key factors: 1) varying FPS requirements and 2) the fluctuating number of traffic signs in each frame. The first factor demands that the system adapts to flexible processing speed requirements, while the second necessitates maintaining a stable processing speed under changing conditions. Note that FPS is a variant of latency requirements. Therefore, we conduct outdoor experiments to evaluate LeapNet-2's ability to adapt to the dynamic conditions to satisfy the system requirements.




(a) (b)

Fig. 20. True frame per second (FPS) and selected layers' ratio for LeapNet-2 under different FPS settings.




(a) (b)

Fig. 21. FPS under different traffic signs' number settings. 30 FPS for yolo-tiny (a) and 10 FPS for yolo (b).

First, we test LeapNet-2 under different FPS settings with a real-world video consisting of 200 continuous frames. We utilize YOLO-tiny as the detector model and ResNet18 as LeapNet-2's target model, resizing frames to 608 × 544 before detection. Fig. 20 illustrates the measured FPS and the number of selected layers for LeapNet-2 under different FPS settings. In Fig. 20a, the measured FPS closely matches the required FPS, reaching the system's minimum (24 FPS) and maximum (47 FPS) achievable performance when the required FPS settings are 20 and 50, respectively. Fig. 20b further supports this observation. When the required FPS is set to 20, LeapNet-2 retains all the layers of ResNet18 to maximize the inference latency of the classifier. Conversely, when the required FPS is set to 50, LeapNet-2 skips all the selectable layers of ResNet18 to minimize inference latency, retaining only a convolutional layer and a fully connected layer to ensure baseline performance.

Furthermore, we evaluate the impact of the number of objects per frame on the actual FPS. To assess system limits, we send a fixed number of objects per frame to the classifier, serving as a stress test for LeapNet to process objects beyond just traffic signs. Fig. 21 shows the measured FPS and the layer retention ratio for LeapNet-2 across different average object counts. Fig. 21a employs settings identical to Fig. 20, while Fig. 21b uses YOLO with an input size of 832 × 704. The target FPS is set at 30 in Fig. 21a and 10 in Fig. 21b, adjustable based on embedded system requirements. The results show that the FPS decreases significantly as the average number of objects increases. For instance, at an average of 5 objects per frame, the FPS drops to 13.8, less than half of the required FPS, as shown in Fig. 21a. In contrast, LeapNet-2 maintains stable FPS across varying object counts as required. Besides, LeapNet-2 achieves slightly lower FPS performance than ResNet18 when the object count is low, due to a minor overhead (<3%) introduced by its decision network.

## 7 Related Works

In this section, we first review related works about adversarial defense, including deterministic and dynamic defense. Next, we discuss the adaptation of DNNs for embedded devices.

■ **Deterministic Defense.** Several strategies have been proposed to address security concerns regarding adversarial examples that can mislead DNN models. Adversarial training [15] includes adversarial examples with correct labels in the training dataset. By iteratively training the DNN on these examples, the model enhances its robustness against adversarial attacks. Several image processing methods also mitigate the influence of adversarial attacks, like feature-squeezing [16]. This method reduces the dimensionality of the input space to squeeze out unnecessary features by decreasing each pixel's color bit depth and applying spatial smoothing. Besides, static ensemble [42] uses an ensemble of models to make joint decisions, enabling robust classification. Gradient masking [17] alters the gradients of the target DNN to obstruct the effectiveness of gradient-based attacks. However, the above mechanisms are deterministic, making them vulnerable to adaptive attacks where adversaries can learn the defense strategies and develop new attack iterations.

Table 4. Comparisons with existing defense methods.

| | Blockdrop[23] | BaRT[18] | Adaptivenet[26] | Sardino[2] | LeapNet-1 | LeapNet-2 |
|---|---|---|---|---|---|---|
| Dynamic defense | ✗ | ✓ | ✗ | ✓ | ✓ | ✓ |
| Computation-efficient | ✓ | ✗ | ✓ | ✗ | ✓ | ✓ |
| Latency-aware | ✗ | ✗ | ✓ | ✓ | ✗ | ✓ |
| Real-time adaption | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ |

■ **Dynamic Defense.** Dynamic defense strategies are proposed to address this issue by maintaining ambiguity, making it harder for the attacker to identify the specific defense for each input, thereby complicating targeted attacks. Stochastic pre-processing defenses [18] apply randomized transformations to the input. BaRT [18] builds on this approach by stochastically combining numerous individual transformations into a randomized barrage, thereby establishing a robust defense. Besides, SAP [19] introduces randomness into the activation function. However, these methods rely on weak stochasticity, making them vulnerable to attacks targeting the entire set of random functions [2]. Dynamic ensembles capture broader variability by leveraging predictions from multiple models [2]. However, combining multiple models incurs huge extra computational overheads, impractical for resource-constrained embedded devices. Our proposed LeapNet counteracts adaptive attacks by dynamically adapting the layer routes while reducing computational redundancy.

■ **DNNs Adaption for embedded Systems.** Recent studies focus on efficient model deployment for embedded scenarios to satisfy resource constraints, as most DNN models are computationally heavy. Model compression methods, including pruning [20] and quantization [21], shrink model sizes for lower storage and computation costs. Other model scaling methods, like early exit [22], layer routing [23], and width-scaling [24], simplify model structures while maintaining accuracy. However, such designs often struggle to meet resource or latency budgets, frequently exceeding limits or causing resource wastage, making them unsuitable for dynamic scenarios. Given that manually designing models for such dynamic environments is time-consuming and inflexible, automated model adaptation has raised research attention. NAS [26, 28], as a relevant model generation solution, attempts to discover architectures that meet specific requirements. However, even recent methods like AdaptiveNet [26] require minutes to update the model, making real-time adaptation infeasible. Besides, Sadino [2] introduces a real-time dynamic adaption method by adjusting ensemble size. However, its scalability is limited to small DNN models due to the use of multiple models for inference, making it impractical for resource-constrained devices. LeapNet addresses these by dynamically adjusting layer routes during inference to adapt to evolving demands in time-series tasks, achieving efficient and responsive adaptation for embedded scenarios.

## 8 Conclusion

We propose LeapNet, comprising two variants, for resource-constrained embedded vision to maintain robustness against adversarial adaptive attacks and real-time adaptation for embedded deployment. LeapNet-1 learns the optimal layer routing distribution and dynamic samples layer routes for inference to enhance robust accuracy while preserving clean accuracy and reducing computational redundancy. LeapNet-2 further optimizes the learned optimal layer routing upon LeapNet-1 to adapt to the dynamic embedded environment under time-series settings. We conduct extensive experiments on various popular datasets and real-world outdoor scenarios to evaluate LeapNet. The effectiveness of LeapNet-1 and LeapNet-2 is demonstrated in comparison with recent static and dynamic defense methods. Moreover, LeapNet-2 can flexibly adapt the layer routes to meet the evolving demands of the time-series embedded vision system, achieving real-time adjustments.

## References


[1] Di Liu, Hao Kong, Xiangzhong Luo, Weichen Liu, and Ravi Subramaniam. Bringing ai to edge: From deep learning's perspective. *Neurocomputing*, 485:297–320, 2022.

[2] Qun Song, Zhenyu Yan, Wenjie Luo, and Rui Tan. Sardino: Ultra-fast dynamic ensemble for secure visual sensing at mobile edge. In *Proceedings of the 2022 International Conference on Embedded Wireless Systems and Networks (EWSN)*, pages 24–35, 2022.

[3] Donghwa Kang, Seunghoon Lee, Cheol-Ho Hong, Jinkyu Lee, and Hyeongboo Baek. Batch-mot: Batch-enabled real-time scheduling for multi-object tracking tasks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024. Presented at The ACM SIGBED International Conference on Embedded Software (EMSOFT).

[4] Jiaming Qiu, Ruiqi Wang, Ayan Chakrabarti, Roch Guérin, and Chenyang Lu. Adaptive edge offloading for image classification under rate limit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(11):3886–3897, 2022. Presented at The ACM SIGBED International Conference on Embedded Software (EMSOFT).

[5] Xisheng Li, Ye Ma, Yuting Chen, Jinghao Sun, Wanli Chang, Nan Guan, Liming Chen, and Qingxu Deng. Priority optimization for autonomous driving systems to meet end-to-end latency constraints. In *2024 IEEE Real-Time Systems Symposium (RTSS)*, pages 402–414. IEEE, 2024.

[6] Bashima Islam and Shahriar Nirjon. Zygarde: Time-sensitive on-device deep inference and adaptation on intermittently-powered systems. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(3):1–29, 2020.

[7] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[8] Meng Shen, Zelin Liao, Liehuang Zhu, Ke Xu, and Xiaojiang Du. Vla: A practical visible light-based attack on face recognition systems in physical world. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 3(3):1–19, 2019.

[9] Brian Testa, Yi Xiao, Harshit Sharma, Avery Gump, and Asif Salekin. Privacy against real-time speech emotion detection via acoustic adversarial evasion of machine learning. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 7(3):1–30, 2023.

[10] Ahmet Soyyigit, Shuochao Yao, and Heechul Yun. Valo: A versatile anytime framework for lidar-based object detection deep neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(11):4045–4056, 2024.

[11] Yuhang Xu, Zixuan Liu, Xinzhe Fu, Shengzhong Liu, Fan Wu, and Guihai Chen. Flex: Adaptive task batch scheduling with elastic fusion in multi-modal multi-view machine perception. In *2024 IEEE Real-Time Systems Symposium (RTSS)*, pages 294–307. IEEE, 2024.

[12] The evolving safety and policy challenges of self-driving cars, 2024. https://www.brookings.edu/articles/the-evolving-safety-and-policy-challenges-of-self-driving-cars/.

[13] Tyler Ward. Areas of improvement for autonomous vehicles: A machine learning analysis of disengagement reports. In *2024 4th Interdisciplinary Conference on Electrics and Computer (INTCEC)*, pages 1–6. IEEE, 2024.

[14] Kevin Eykholt, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning visual classification. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1625–1634, 2018.

[15] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.

[16] Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. In *Proceedings 2018 Network and Distributed System Security Symposium*. Internet Society, 2018.

[17] Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *International conference on machine learning*, pages 274–283. PMLR, 2018.

[18] Edward Raff, Jared Sylvester, Steven Forsyth, and Mark McLean. Barrage of random transforms for adversarially robust defense. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6528–6537, 2019.

[19] Guneet S Dhillon, Kamyar Azizzadenesheli, Zachary C Lipton, Jeremy D Bernstein, Jean Kossaifi, Aran Khanna, and Animashree Anandkumar. Stochastic activation pruning for robust adversarial defense. In *International Conference on Learning Representations*, 2018.

[20] Aojun Zhou, Yang Li, Zipeng Qin, Jianbo Liu, Junting Pan, Renrui Zhang, Rui Zhao, Peng Gao, and Hongsheng Li. Sparsemae: Sparse training meets masked autoencoders. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16176–16186, 2023.

[21] Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1972–1981, 2023.

[22] Nafiul Rashid, Berken Utku Demirel, Mohanad Odema, and Mohammad Abdullah Al Faruque. Template matching based early exit cnn for energy-efficient myocardial infarction detection on low-power wearable devices. *Proceedings*

*of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 6(2):1–22, 2022.

[23] Zuxuan Wu, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S Davis, Kristen Grauman, and Rogerio Feris. Blockdrop: Dynamic inference paths in residual networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8817–8826, 2018.

[24] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. Nimble: Efficiently compiling dynamic neural networks for model inference. *Proceedings of Machine Learning and Systems*, 3:208–222, 2021.

[25] Ionel Gog, Sukrit Kalra, Peter Schafhalter, Joseph E Gonzalez, and Ion Stoica. D3: a dynamic deadline-driven approach for building autonomous vehicles. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 453–471, 2022.

[26] Hao Wen, Yuanchun Li, Zunshuai Zhang, Shiqi Jiang, Xiaozhou Ye, Ye Ouyang, Yaqin Zhang, and Yunxin Liu. Adaptivenet: Post-deployment neural architecture adaptation for diverse edge environments. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–17, 2023.

[27] Stopping distances: Vehicle speed, crash risk, thinking and braking time, 2016. https://www.qld.gov.au/transport/safety/road-safety/driving-safely/stopping-distances.

[28] Qinsi Wang and Sihai Zhang. Dgl: device generic latency model for neural architecture search on mobile devices. *IEEE Transactions on Mobile Computing*, 2023.

[29] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[30] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: a multi-class classification competition. In *The 2011 international joint conference on neural networks*, pages 1453–1460. IEEE, 2011.

[31] Nicolai Wojke and Alex Bewley. Deep cosine metric learning for person re-identification. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 748–756. IEEE, 2018.

[32] Dji mavic air 2 user manual, 2020.05. https://dl.djicdn.com/downloads/Mavic_Air_2/20200511/Mavic_Air_2_User_Manual_v1.0_en.pdf.

[33] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[34] Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E Gonzalez. Skipnet: Learning dynamic routing in convolutional networks. In *Proceedings of the European conference on computer vision (ECCV)*, pages 409–424, 2018.

[35] Radu Timofte, Karel Zimmermann, and Luc Van Gool. Multi-view traffic sign detection, recognition, and 3d localisation. *Machine vision and applications*, 25:633–647, 2014.

[36] scikit-learn, 2024. https://scikit-learn.org/stable/.

[37] Alexey Kurakin, Ian J Goodfellow, and Samy Bengio. Adversarial examples in the physical world. In *Artificial intelligence safety and security*, pages 99–112. Chapman and Hall/CRC, 2018.

[38] Maksym Andriushchenko, Francesco Croce, Nicolas Flammarion, and Matthias Hein. Square attack: a query-efficient black-box adversarial attack via random search. In *European conference on computer vision*, pages 484–501. Springer, 2020.

[39] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International conference on machine learning*, pages 2206–2216. PMLR, 2020.

[40] Yolo: Real-time object detection. https://pjreddie.com/darknet/yolo/.

[41] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.

[42] Thilo Strauss, Markus Hanselmann, Andrej Junginger, and Holger Ulmer. Ensemble methods as a defense to adversarial perturbations against deep neural networks. *arXiv preprint arXiv:1709.03423*, 2017.

**Response Letter to Reviewers**

Dear Reviewers,
We sincerely thank you for your valuable feedback. We have carefully taken each comment into account and revised our previous manuscript accordingly. To facilitate the review process, we have highlighted the newly added or revised parts in blue in the above manuscript.

## Meta Review:

**Comment 1:** *Provide more details on latency-aware routing and analyze runtime/memory overhead.*
**Response:** We would like to thank the reviewer for this valuable comment. Following the reviewers' comments, we have added more discussion on the latency-aware routing mechanism in Section 4.2 on Page 13$\tilde{1}$4. Additionally, we have included further experimental results and analysis on both runtime and memory overhead for training and inference stages in Section 3.3 on Page 10.

**Comment 2:** *Evaluate (at least qualitatively) against stronger/adaptive attacks to better demonstrate robustness, and discuss additional defenses beyond Blockdrop to highlight novelty and effectiveness.*
**Response:** We would like to thank the reviewer for this valuable comment. We also agree with the reviewer regarding this comment. Following the reviewers' comments, we have added stronger adaptive attacks to demonstrate the robustness of our methods. To clarify, in the previous manuscript, we have included three defense baselines, including two deterministic defense methods, adversarial training and blockdrop, and a dynamic defense, Sardino, in the experiment section. Following the comments of reviewer #C, we have added an input transformation-based dynamic defense method, BaRT.

**Comment 3:** *Include more models and datasets like MobileNet or EfficientNet and more realistic benchmarks.*
**Response:** We would like to thank the reviewer for this valuable comment. We also agree with the reviewer regarding this comment. In the previous manuscript, we mainly considered small to mid-size models (ResNet18, ResNet34 and ResNet101). Therefore, we have added a more complex model, EfficientNet, in the experiment section and discussed the possibility of applying the current methods to transform-based large models. Besides, we also test our method on a more complex dataset, tiny-imagenet, with 200 classes.

**Comment 4:** *Clarify relevance to embedded systems by justifying hardware choices and emphasizing constraints like latency and memory.*
**Response:**

## Reviewer #A:

**Comment 1:** *Limited Attack Evaluation: The defense is evaluated only against standard attacks like PGD and FGSM; no experiments with stronger or adaptive attacks (e.g., AutoAttack) are provided.*
**Response:**We would like to thank the reviewer for this valuable comment. We have addressed this issue in our response to Comment 2 of the Meta review. Please refer to that reply for details.

**Comment 2:** *Narrow Model and Dataset Scope: Experiments are limited to small to mid-size models and standard datasets (e.g., CIFAR-10), with no evaluation on larger real-world models or more complex datasets.*
**Response:**

## Reviewer #B:

**Comment 1:** *The paper is very weakly related to embedded systems. The authors evaluate their solution on "an edge server with Nvidia RTX2080Ti with 11GB memory, and two embedded devices, a Jetson Orin with 64 GB memory and a Jetson Xavier with 16". I would argue that the Jetson devices are very powerful purpose-built ML inference systems, far more capabable than most CPUs (not GPUs). I see no real connection to embedded systems apart from the 5ms goal for inference. This goal also appears to only be barely met. In my opinion, the paper is better suited for a (applied) ML-oriented venue.*
**Response:** Thank you for the comment. We have added clarification in Section 3.2.

**Comment 2:** *Performance discussion is made wrt to ResNet18, but I am missing convincing arguments that ResNet18 would actually be applied in the embedded context presented in the paper. The solution might be more efficient than ResNet18, but that does not mean that it is suitable for the claimed application scenario. Notably, Edge Servers would not be used for object detection in real-time systems (as their availability is likely not guaranteed).*
**Response:**

**Comment 3:** *"[19, 20].."*
**Response:** We would like to thank the reviewer for this valuable comment. We have revised this.

**Comment 4:** *why mix metric and imperial (" 6 meters at 20 mph"), this seems UK/Canada specific? or is mph meters per hour?*
**Response:** We would like to thank the reviewer for this valuable comment. We agree that mixing metric and imperial units may cause confusion. We have standardized all measurements to the metric system for consistency and clarity. Additionally, since the previous news link was no longer accessible, we have replaced it with a new and reliable news source with different speed settings.

## Reviewer #C:

**Comment 1:** *In LeapNet-1, the key design, in my understanding, is the stochasticity to counteract adaptive attack. Therefore, more details on the rationale and design of the defense against adaptive attack should be elaborated.*
**Response:** Thank you for the comment. We have added clarification in Section 3.2.

**Comment 2:** *While the paper effectively demonstrates the weaknesses of deterministic defenses, it primarily compares LeapNet against one specific deterministic defense method (Blockdrop). A more comprehensive evaluation against a wider range of state-of-the-art defense mechanisms (both deterministic and dynamic) would further strengthen the claims. For example, the authors could have compared LeapNet with adversarial training techniques or input transformation methods to provide a broader understanding of its relative strengths and weaknesses.*
**Response:**

**Comment 3:** *The paper primarily focuses on ResNet as the target model. Although the authors claim that LeapNet can be extended to other DNN architectures, the experimental results and analysis are limited to ResNet. Further investigation or discussion is needed to validate the effectiveness and generalizability of LeapNet across different network architectures, such as MobileNet or EfficientNet, which are commonly used in embedded applications.*
**Response:**

**Comment 4:** *A small typo: "[19, 20].." on page 2.*
**Response:** We would like to thank the reviewer for this valuable comment. We have revised this.

**Comment 5:** *How are the latency constraints determined? Especially in dynamic and heterogeneous environments.*
**Response:**

**Comment 6:** *What if there are multiple layer routings that satisfy the latency constraint?*
**Response:**

**Comment 7:** *A more detailed discussion of the implementation complexity and overhead would be valuable. While the authors mention the small size of the decision network, a thorough analysis of the computational cost associated with the latency predictor and the two-stage training process would give readers a clearer understanding of the practical considerations for deploying LeapNet.*
**Response:** We would like to thank the reviewer for this valuable comment. We also agree with the reviewer regarding this comment. We added a comprehensive analysis

**Comment 8:** *Additionally, exploring the limitations of LeapNet in extreme dynamic conditions or under very tight resource constraints could provide valuable insights for future research.*
**Response:**

## Reviewer #D:

**Comment 1:** *The dynamic layer routing and real-time adaptation mechanisms might introduce additional complexity in implementation and deployment, which could be challenging for some embedded systems with limited memory resources. The paper does not address the memory overhead required by adding the dynamic routing layers. Memory utilization is an important metric for a lot of resource-constrained devices.*
**Response:** We would like to thank the reviewer for this valuable comment. We also agree with the reviewer regarding this comment. In the previous manuscripts, we neglected the analysis of memory cost. However, the size of decision network is very limited compared to the target model and costs very limited static memory. Besides, the added experiments showed that LeapNet costs less memory during inference since LeapNet activated fewer intermediate features maps during inference. And the details can be found in section 3.3.

**Comment 2:** *LeapNet-2 shows slightly lower accuracy compared to LeapNet-1 due to sacrificing some performance for latency adaptation. This entails that a trade-off exists between security and efficiency. The paper would benefit from better exploring this tradeoff and comment on how to navigate such tradeoff.*
**Response:** We would like to thank the reviewer for this valuable comment. We also agree with the reviewer regarding this comment.

**Comment 3:** *The paper will benefit from showing the proposed methodology over multiple applications to assess generalizability.*
**Response:**